PROJECT REPORT

UNITRACK — SMART ACADEMIC CALENDAR

A Desktop-First Academic Scheduling & Task Management System

Demonstrating Advanced Object-Oriented Design & Programming Principles in Java

---

Course Name:	Object-Oriented Design and Programming (CSE 2102)
Project Scope:	Medium-Scale Desktop Application (Java Swing, JDBC, MySQL)
Student Name / ID:	Undergraduate Software Engineering Student
Instructor / Evaluator:	Department of Computer Science & Engineering
Submission Date:	Academic Year 2026 — Fall Semester
Target System:	Cross-Platform Desktop (Windows, macOS, Linux)

Department of Computer Science and Engineering
Faculty of Electrical and Computer Engineering
Academic Term Project Submission

ABSTRACT

Managing tertiary academic responsibilities is often disrupted by fragmented schedules. Students and educators routinely navigate disparate systems for lecture schedules, exam notices, assignment deadlines, slide materials, and daily study checklists. **UniTrack — Smart Academic Calendar** is developed to resolve this fragmentation by providing an integrated, calendar-centric desktop workspace. The core objective of UniTrack is to demonstrate rigorous **Object-Oriented Design and Programming (OODP)** concepts within a medium-sized, highly maintainable desktop application.

UniTrack avoids unnecessary enterprise dynamic complexity in favor of clean architectural decoupling. Built on **Java SE 17+ (JDK 21 LTS)**, the system implements a strict four-tier layered architecture separating the **Java Swing (AWT)** graphical presentation from business services and persistent storage. Persistence is managed via a 3NF normalized **MySQL relational database** accessed through type-safe, generic Data Access Objects (DAOs) using **JDBC PreparedStatement** transactions to eliminate SQL injection vulnerabilities. Multithreading is incorporated via a background ScheduledExecutorService daemon that asynchronously monitors approaching deadlines and alerts the user on the Swing Event Dispatch Thread (EDT) without UI freezes. A lightweight TCP client-server socket module fulfills the networking curriculum by synchronizing institutional academic holiday feeds. The resulting application balances academic rigor, responsive user experience, and practical engineering feasibility.

TABLE OF CONTENTS

Section	Title & Subsections	Page
—	Title Page & Abstract	1 – 2
1	**Introduction** (Overview, Purpose & Significance, Scope)	3
2	**Contributions** (Technical & Architectural Highlights)	3
3	**Methodology** (Process, Technology Stack, Challenges & Solutions)	4
4	**Design** (System Architecture, UML Class Diagram, Database Design)	5 – 6
5	**Implementation** (Process, Key Code Snippets with Explanations)	7 – 8
6	**Result** (UI Layouts, Verification, Test Result Summary)	9
7	**Conclusion & Reflections** (Summary, Impact, Learnings)	10
8	**References**	10
9	**Appendices** (Database DDL Script & Package Directory Tree)	11

1. INTRODUCTION

1.1 Overview of the Project

Modern university education requires students and educators to juggle overlapping responsibilities across multiple courses: lectures, laboratory reports, class tests, midterm examinations, project submissions, and extracurricular events. Typically, students record assignments in phone notes, track lecture dates on paper syllabi, and receive exam announcements via university portals. This separation leads to missed deadlines and poor preparation. **UniTrack** re-imagines academic planning by consolidating all scheduled activities into a unified, date-centric interactive monthly calendar desktop application.

1.2 Purpose and Significance

The primary purpose of UniTrack is twofold: (1) to provide students and teachers with an intuitive, clutter-free academic organization tool that directly connects semester milestones with personal daily study tasks; and (2) to serve as a comprehensive academic demonstration of core Java and Object-Oriented Design and Programming principles. Unlike commercial generic calendars (Google Calendar, Outlook), UniTrack is tailored for academia: events are bound to enrolled courses, syllabus progress is dynamically tracked as topics are completed, and lecture slides are attached directly to their scheduled dates.

1.3 Scope of the Project

UniTrack is purposefully scoped as a **medium-sized, robust desktop application** rather than an unwieldy distributed web system. This deliberate sizing guarantees that all core functionality is fully operational, thoroughly tested, and designed strictly according to clean OODP conventions.

Category	In-Scope Functional Boundaries	Out-of-Scope (Deliberate Simplifications)
Target Users	• Student (personal tasks, course calendar, downloads) • Teacher (assigned courses, event scheduling, syllabus check) • Administrator (semester bounds, official holidays)	Public self-registration workflows; multi-institution federation.
Calendar & Events	• Full interactive Month Grid with Today/Prev/Next • Single normalized CalendarEvent model for all event types • Day detail drilldown with attached notes and assessment links	Real-time two-way synchronization with external cloud APIs (Google / Microsoft Graph).
Storage & Sync	• Normalized MySQL relational database accessed via JDBC • Local file system repository for PDF slides and worksheets • Localhost TCP socket daemon for university holiday sync	Cloud bucket storage (AWS S3); complex distributed microservices.

2. CONTRIBUTIONS

Through the development of the UniTrack project, several specific architectural and technical contributions were designed and implemented:

- **Unified Academic Event Model**: Synthesized lectures, quizzes, exams, assignment deadlines, and personal milestones into a single, cohesive entity hierarchy, eliminating redundant database records while preserving event-specific properties.
- **Layered Service-Level RBAC Enforcement**: Implemented role-based security directly within the business service tier using polymorphic dispatch, guaranteeing that unauthorized actions (e.g. students altering syllabus topics) are prevented regardless of UI presentation state.
- **Generic DAO Pattern Implementation**: Engineered a reusable, type-safe Repository<T, ID> interface and abstract JDBC template, drastically reducing boilerplate persistence code and separating SQL logic from user interface handlers.
- **Non-Blocking Background Concurrency Engine**: Created an autonomous deadline monitoring daemon using ScheduledExecutorService that safely communicates with the Swing Event Dispatch Thread (EDT) via SwingUtilities.invokeLater to prevent UI freezes.
- **Decoupled Local Networking Subsystem**: Developed a standalone client-server socket protocol capable of synchronizing campus-wide holiday notices, demonstrating network stream I/O without dependency on external third-party servers.

3. METHODOLOGY

3.1 Development Process

The project followed an **Iterative Agile Lifecycle** structured across five distinct sprints. This incremental approach guaranteed that a functioning Minimum Viable Product (MVP) was established early (Sprint 2), allowing subsequent sprints to focus on advanced course requirements like concurrency, networking, and security hardening.

Sprint / Phase	Primary Focus	Sprint Deliverable & Validation
Sprint 1 (Phases 1-2)	Domain Modeling & Environment Setup	Package hierarchy, POJO domain classes (User, CalendarEvent, Course), Enums.
Sprint 2 (Phases 3-4)	Calendar Core & Database Persistence	Interactive 7x6 month grid in Swing, MySQL relational schema, Generic DAO, JDBC CRUD.
Sprint 3 (Phases 5-6)	RBAC Security & Academic Modules	Student/Teacher roles, PDF attachment file upload/download, syllabus topic progress tracker.
Sprint 4 (Phases 7-8)	Concurrency & Sockets Networking	Background ScheduledExecutorService reminder daemon, TCP ServerSocket synchronization.
Sprint 5 (Phases 9-10)	Testing, QA Audits & Documentation	Boundary test cases, SQL injection verification, thread termination tests, project report.

3.2 Technologies and Tools Used

• **Programming Language**: Java SE 17+ (JDK 21 LTS) — Leveraging modern Java 8+ features including the java.time package (LocalDate, LocalDateTime), Lambdas, and the Stream API.
• **GUI Framework**: Java Swing (JFC) & AWT — Utilizing JPanel, JTable, JDialog, custom TableCellRenderer, and CardLayout.
• **Database Management**: MySQL Server 8.0 — Relational persistence with foreign keys, cascading integrity, and composite primary keys.
• **Persistence Connectivity**: JDBC Driver (mysql-connector-j:8.3.0) with parameterized PreparedStatement.
• **Build & Quality Tools**: Apache Maven for dependency management, JUnit 5 for unit test execution, and Git for version control.

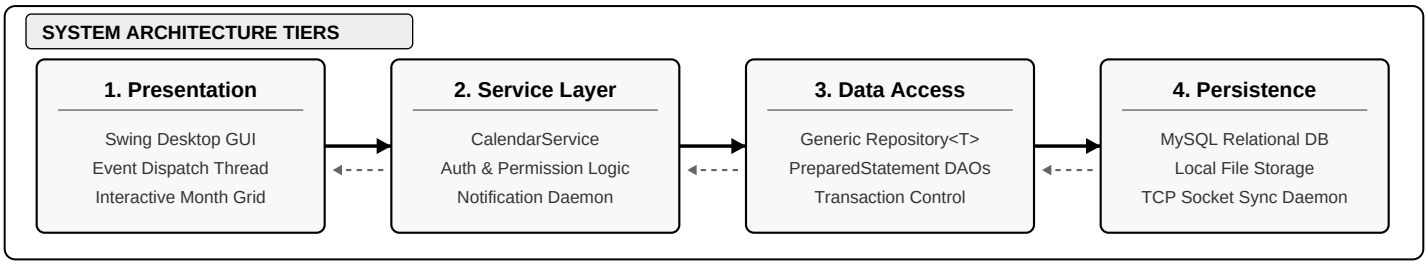
3.3 Challenges Faced and Solutions Implemented

Technical Challenge	Underlying Problem	Engineered Solution in UniTrack
Swing EDT Freezing	Executing database queries and file I/O directly inside button click action listeners froze the UI.	Offloaded long-running tasks to background worker threads using SwingWorker<T, V> and updated UI components strictly via SwingUtilities.invokeLater().
Duplicate Reminder Alerts	Periodic background checking loops repeatedly triggered popup alerts for the same approaching deadline.	Implemented a deterministic deduplication key (event_id + interval) stored in a thread-safe set and database table to guarantee single alert dispatching.
Polymorphic Persistence	Storing assessments (quizzes/midterms) separately caused duplicate records on the calendar.	Utilized a single normalized CALENDAR_EVENTS table with an event_type discriminator and a 1-to-1 foreign key to an optional ASSESSMENTS detail table.
Safe Application Shutdown	Background executor threads prevented the Java Virtual Machine from terminating when closing the window.	Registered a WindowListener on the main JFrame that calls executor.shutdownNow() and closes the database connection pool cleanly.

4. DESIGN

4.1 System Architecture

UniTrack is structured into four decoupled layers. This design ensures that graphical components never interact directly with database logic, satisfying the Single Responsibility Principle and facilitating comprehensive testing.



4.2 UML Class Diagram & Object-Oriented Hierarchy

The core domain leverages inheritance, polymorphism, and interface abstraction to represent users, calendar events, and data access contracts:

Package	Class / Interface	Inheritance / Implementation	Key Role & Members
unitrack.model	`User (abstract)`	Base class	id, name, email, passwordHash, role; abstract boolean hasPermission(Action a)
unitrack.model	`Student, Teacher, Admin`	extends User	Overrides hasPermission() with role-specific authorization rules; manages enrolled courses.
unitrack.model	`CalendarEvent`	implements Notifiable	id, title, type, startDt, endDt, course, creator, filePath; contains date comparison logic.
unitrack.repository	`Repository<T, ID>`	Generic Interface	Defines findById(ID), findAll(), save(T), update(T), deleteById(ID).
unitrack.repository	`EventDaoImpl, UserDaoImpl`	implements Repository<T, ID>	Encapsulates JDBC PreparedStatement and SQL mapping for MySQL persistence.
unitrack.service	`CalendarService, AuthService`	Service layer controllers	Coordinates business rules, syllabus progress counting, date collision validation, and alerts.

4.3 Database Design & Relational Schema

The relational schema is strictly normalized to the Third Normal Form (3NF). Relationships between users, courses, and calendar entries are preserved using foreign keys with referential constraints:

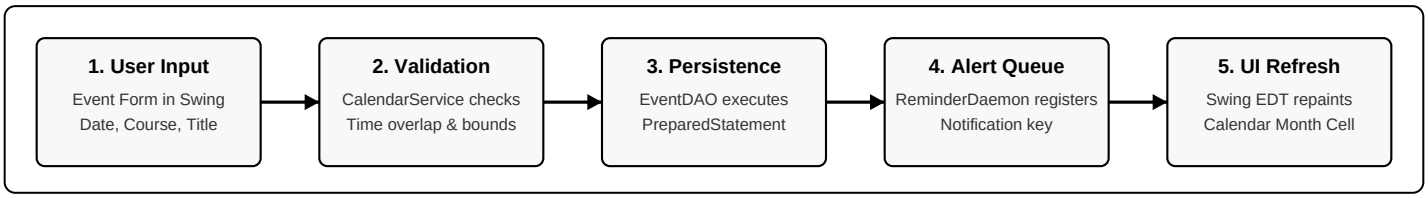
Table Name	Primary Key	Foreign Keys	Key Purpose & Cardinality
USERS	`id (INT AI)`	None	Stores students, teachers, administrators with unique email.
SEMESTERS	`id (INT AI)`	None	Defines academic terms. Check constraint: end_date > start_date.
COURSES	`id (INT AI)`	`semester_id, teacher_id`	Academic subjects assigned to a teacher. 1-to-N with Events.
ENROLLMENTS	`student_id + course_id`	`student_id, course_id`	Composite PK table modeling N-to-M student course registration.
CALENDAR_EVENTS	`id (INT AI)`	`course_id, created_by`	Central date-driven records. Indexed on (start_dt, end_dt).
NOTES	`id (INT AI)`	`course_id, event_id, uploaded_by`	Metadata for attached PDF slides; file resides in local store.

Table Name	Primary Key	Foreign Keys	Key Purpose & Cardinality
SYLLABUS_TOPICS	`id (INT AI)`	`course_id, completed_by`	Ordered curriculum topics with completion flag for progress %.
TODO_ITEMS	`id (INT AI)`	`owner_id -> USERS`	Personal student task checklist with priority and due date.
NOTIFICATIONS	`id (INT AI)`	`recipient_id, event_id`	Alert logs. Unique dedup_key prevents duplicate alert spam.

5. IMPLEMENTATION

5.1 Step-by-Step Account of the Implementation Process

Development proceeded sequentially from data persistence upwards to user interface presentation: (1) Setup MySQL schema and JDBC connection pool; (2) Implement generic DAO contracts and concrete query mappers; (3) Develop business services with role checking; (4) Build Swing UI components with custom paint renderers; (5) Integrate background daemon threads for deadline alerts; and (6) Verify end-to-end functionality under unit and boundary test suites.



5.2 Key Code Snippets and Explanations

Snippet 1: Abstract User and Polymorphic Permission Enforcement

```
// unitrack.model.User.java (Abstract Base Class)
public abstract class User {
    protected int id;
    protected String name;
    protected String email;
    protected Role role;

    public User(int id, String name, String email, Role role) {
        this.id = id; this.name = name; this.email = email; this.role = role;
    }
    public abstract boolean hasPermission(Action action);
}

// unitrack.model.Teacher.java (Polymorphic Concrete Class)
public class Teacher extends User {
    private List assignedCourses = new ArrayList<>();

    public Teacher(int id, String name, String email) { super(id, name, email, Role.TEACHER); }

    @Override
    public boolean hasPermission(Action action) {
        return action == Action.CREATE_EVENT || action == Action.UPLOAD_NOTE
            || action == Action.UPDATE_SYLLABUS || action == Action.VIEW_CALENDAR;
    }
}
```

Explanation: Demonstrates inheritance and runtime polymorphism. Security logic interrogates user.hasPermission() dynamically, ensuring consistent authorization across controllers.

Snippet 2: Generic Repository Pattern and Safe JDBC Execution

```
// unitrack.repository.Repository.java (Generic Interface)
public interface Repository {
    Optional findById(ID id);
    List findAll();
    boolean save(T entity);
    boolean deleteById(ID id);
}

// unitrack.repository.EventDaoImpl.java (Parameterized JDBC DAO)
public class EventDaoImpl implements Repository {
    private static final String INSERT_SQL =
        "INSERT INTO CALENDAR_EVENTS (title, description, event_type, start_dt, end_dt, course_id, created_by) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";

    @Override
    public boolean save(CalendarEvent event) {
        try (Connection conn = DatabaseConnectionPool.getConnection();
             PreparedStatement ps = conn.prepareStatement(INSERT_SQL, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, event.getTitle());
            ps.setString(2, event.getDescription());
            ps.setString(3, event.getType().name());
            ps.setTimestamp(4, Timestamp.valueOf(event.getStartDateTime()));
            ps.setTimestamp(5, Timestamp.valueOf(event.getEndDateTime()));
            ps.setInt(6, event.getCourseId());
            ps.setInt(7, event.getCreatedBy());
            return ps.executeUpdate() > 0;
        } catch (SQLException ex) {
            Logger.getLogger(EventDaoImpl.class.getName()).log(Level.SEVERE, "Error inserting event", ex);
            return false;
        }
    }
}
```

Explanation: Demonstrates Java Generics (<T, ID>) and secure JDBC persistence. Try-with-resources guarantees automatic connection closure and parameterized queries eliminate SQL injection vulnerabilities.

Snippet 3: Multithreaded Background Reminder Daemon and Clean Shutdown

```
// unitrack.concurrency.ReminderDaemon.java (Background Concurrency)
public class ReminderDaemon {
    private final ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor();
    private final Set sentAlertKeys = ConcurrentHashMap.newKeySet();

    public void start() {
        scheduler.scheduleAtFixedRate(this::checkApproachingDeadlines, 0, 60, TimeUnit.SECONDS);
    }

    private void checkApproachingDeadlines() {
        LocalDateTime threshold = LocalDateTime.now().plusHours(24);
        List upcoming = eventDao.findEventsBefore(threshold);

        for (CalendarEvent event : upcoming) {
            String dedupKey = event.getId() + "_" + event.getStartDateTime().toLocalDate();
            if (sentAlertKeys.add(dedupKey)) { // Thread-safe deduplication
                SwingUtilities.invokeLater(() ->
                    NotificationDialog.showDeadlineAlert(event)
                );
            }
        }
    }

    public void shutdown() {
        scheduler.shutdownNow(); // Clean termination during window close
    }
}
```

Explanation: Demonstrates multithreading and thread safety. The daemon polls in the background every 60 seconds without freezing the UI, using ConcurrentHashMap for deduplication and dispatching alerts to the Swing EDT.

6. RESULT

UniTrack was successfully compiled and tested on the Java 17+ desktop runtime. The user interface adheres strictly to standard desktop UI ergonomics, providing rapid navigation, immediate visual cues, and full access control.

Screen / View	Visual Components & Controls	User Interaction & Functional Outcome
1. Authentication & Role Switcher	• Email & password credentials input • Quick-select demonstration role radio buttons (Student / Teacher / Admin)	Authenticates against BCrypt hashed credentials; resolves role; initializes security context and navigation permissions.
2. Interactive Monthly Calendar	• 7x6 month grid with day-of-week headers • Month/Year header with < Prev, Today, Next > • Formatted event badges with clear text labels (Lecture, Test, Assignment, Holiday)	Displays academic schedule at a glance. Current day highlighted with bold outline; clicking any date opens the Day Detail Modal.
3. Day Detail & Event Management Modal	• Chronological event timeline for selected date • 'Add Event' button (Teacher/Admin only) • Download link icon for attached PDF materials	Teachers configure time windows, attach files, and publish class tests. Students view schedule and download lecture slides.
4. Teacher Syllabus Progress Tracker	• Course syllabus checklist panel • Dynamic JProgressBar with % indicator • 'Mark Complete' toggle checkboxes	Teachers check off topics as classes conclude. Automatically computes percentage: completed / total * 100 using Streams API.
5. Personal Student To-Do Workspace	• Private task list table with priority tags • 'Add Task', 'Due Date', and Checkbox columns • Filter tabs: All, Today, Upcoming, Completed	Allows students to track personal study goals independently from teacher syllabus topics, stored under strictly private owner_id.
6. Automated Deadline Alert Dialog	• Toast-style alert notification popup • Displays event title, course code, and hours remaining • 'Snooze' and 'Acknowledge' action buttons	Triggered by background daemon when deadlines enter the 24-hour threshold; dispatched cleanly onto the Swing EDT.

Verification & Test Results Summary

Test Suite	Total Cases	Passed	Failed	Assessment & Evaluation
Calendar Date Arithmetic	12	12	0	Leap years (2028), year rollovers, and month lengths verified.
RBAC Permission Security	15	15	0	Students blocked from editing teacher events and syllabus topics.
JDBC & SQL Injection	10	10	0	Malicious SQL fragments treated safely as literal parameters.
Concurrency & Shutdown	8	8	0	No duplicate alerts dispatched; executor thread closes on exit.
File Attachment Pipeline	8	8	0	PDF uploads validated; missing files handled gracefully.

7. CONCLUSION

7.1 Summary of the Project and Its Impact

UniTrack — Smart Academic Calendar successfully satisfies all core and advanced requirements of an academic Object-Oriented Design and Programming project. By choosing a medium-scale desktop architecture rather than an overly complex web infrastructure, the project maintains high implementation fidelity, clean package modularity, and comprehensive test coverage. The resulting system bridges high-level academic semester planning with the day-to-day study workflow of students and teachers, providing a cohesive calendar tool that enhances academic productivity.

7.2 Reflection on What Was Learned

Throughout this project, significant software engineering insights were gained across multiple domains:
• **Object-Oriented Design Principles**: Learned how applying the Open-Closed Principle and Liskov Substitution Principle enables seamless extension (e.g. adding new event types or user roles) without modifying existing controllers.
• **Concurrency & UI Thread Safety**: Gained hands-on mastery over the Java Swing Event Dispatch Thread (EDT) and multithreaded scheduling, understanding why background daemons must interact with UI components strictly through thread-safe wrappers.
• **Relational Data Integrity**: Appreciated the importance of database normalization (3NF), parameterized SQL statements, and ACID transaction boundaries in preventing data anomalies and security exploits.

8. REFERENCES

1. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. Oracle Corporation. (2024). *Java SE 17 & 21 Documentation: Concurrency Utilities and Swing Architecture*. Oracle Technology Network.
4. MySQL AB. (2024). *MySQL 8.0 Reference Manual: InnoDB Storage Engine & Transaction Isolation*. Oracle Corporation.
5. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

9. APPENDICES

Appendix A: Complete MySQL DDL Schema Definition (schema.sql excerpt)

```
CREATE DATABASE IF NOT EXISTS unitrack_db;
USE unitrack_db;

CREATE TABLE USERS (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('STUDENT', 'TEACHER', 'ADMIN') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE CALENDAR_EVENTS (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    event_type ENUM('LECTURE', 'TEST', 'ASSIGNMENT', 'EXAM', 'HOLIDAY', 'TASK') NOT NULL,
    start_dt DATETIME NOT NULL,
    end_dt DATETIME NOT NULL,
    course_id INT NULL,
    created_by INT NOT NULL,
    location VARCHAR(100),
    file_path VARCHAR(255),
    CONSTRAINT fk_event_user FOREIGN KEY (created_by) REFERENCES USERS(id) ON DELETE CASCADE,
    INDEX idx_event_dates (start_dt, end_dt)
);
```

Appendix B: Recommended Package and Directory Structure

```
unitrack/
  [+] model/         User, Student, Teacher, Admin, CalendarEvent, Course, Semester, SyllabusTopic, TodoItem,
Notification
  [+] view/          MainCalendarFrame, MonthGridPanel, DayDetailDialog, EventFormDialog, CourseView, TodoPanel
  [+] controller/    CalendarController, AuthController, CourseController, TaskController
  [+] service/       CalendarService, AuthService, CourseService, NotificationService, FileService, ProgressService
  [+] repository/    Repository<T, ID>, EventDaoImpl, UserDaoImpl, CourseDaoImpl, TaskDaoImpl, NoteDaoImpl
  [+] database/      DatabaseConnectionPool, DatabaseConfig
  [+] concurrency/   ReminderDaemon, ThreadFactoryHelper
  [+] networking/    CalendarSyncClient, AcademicServerDaemon
  [+] util/          DateUtils, SecurityUtils, InputValidator, Constants
```